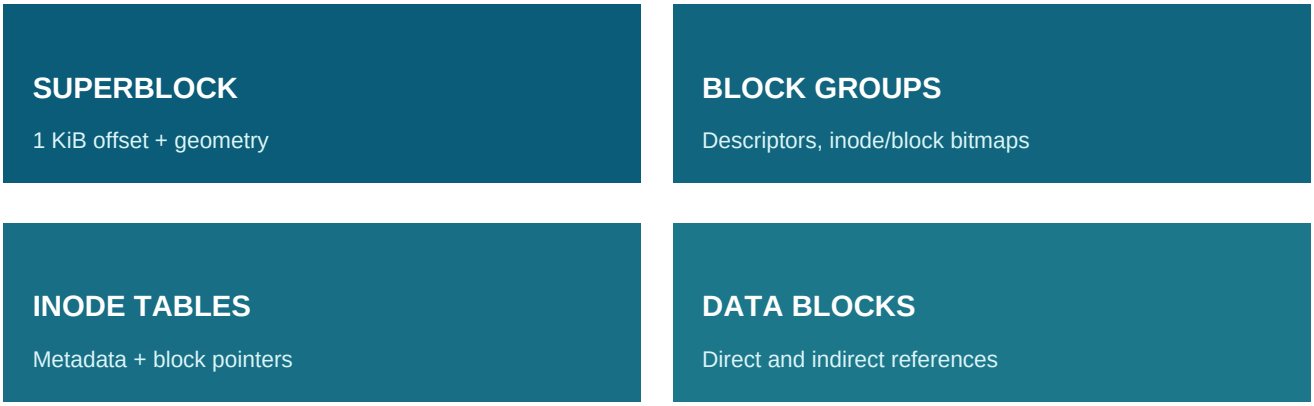
EXT2 - Second Extended Filesystem

HISTORY / ORIGINAL DESIGN / DISK FORMAT

The 1993 Linux filesystem that established extensible inode, block group and bitmap structures later reused by ext3 and ext4.

This is a technical reference to the upstream filesystem's historical design, not an installer manual. The format's on-media semantics are distinct from the independent InfiltratorOS module, its APIs and any Linux kernel adapter.

CONCEPTUAL MEDIA MAP



CONTENTS

1. Historical chronology
2. Original on-disk architecture
3. Allocation and directories
4. Integrity and recovery
5. Design rationale and limits
6. InfiltratorOS integration boundary

EXT2 - Second Extended Filesystem

HISTORICAL CHRONOLOGY

Linux initially used the Minix filesystem, whose filename and size constraints became restrictive. The first extended filesystem followed, and the Second Extended Filesystem was released in January 1993. Linux documentation attributes the rewrite to Remy Card, Theodore Ts’o and Stephen Tweedie. Ext2 supplied a more Unix-like filesystem suited to the expanding use of Linux on larger disks, with better allocation policy and metadata capabilities.

Ext2 became the dominant Linux disk format during the 1990s. Its design supported later compatible evolution: ext3 added journalling, and ext4 added extents and modern feature flags. Ext2 without journalling remained useful for certain flash media, boot areas and constrained devices, but it does not inherit the safety properties of ext3 or ext4 just because their format ancestry is shared.

EXT2 - Second Extended Filesystem

ORIGINAL ON-DISK ARCHITECTURE

The primary superblock begins 1024 bytes after the filesystem start, not always at the first sector boundary. It records block counts, free counts, inode counts, block size, per-group geometry, compatible/incompatible/readonly-compatible feature flags and volume state. Block group descriptors locate their bitmaps and inode tables; backups of important metadata are placed according to revision and sparse-super rules.

Each inode records file mode, ownership, timestamps, link count, size and block references. Traditional ext2 block pointers use direct, single-indirect, double-indirect and triple-indirect addressing, allowing progressively larger files. The inode data mapping is separate from the directory that names the file. Correct interpretation requires matching inode size, directory entry revision and enabled feature flags.

EXT2 - Second Extended Filesystem

ALLOCATION AND DIRECTORIES

Block groups are intended to keep related file data, inodes and directories close on rotating media. Bitmaps represent allocated blocks and inodes, avoiding a single FAT-like allocation chain. A file’s inode points to data blocks or indirect blocks, and free-space search uses group-local information. The physical placement strategy is a policy choice, not an intrinsic guarantee of contiguity.

Directories store inode-number and name pairs as variable-length records that must be bounded by record lengths and block sizes. Hard links are multiple directory names referencing one inode; their link counts matter for recovery. Symbolic links can store small targets inside an inode. Unix permission bits and file types live in inode metadata, unlike simple DOS file attributes.

EXT2 - Second Extended Filesystem

INTEGRITY AND RECOVERY

The original ext2 format has no metadata journal. A crash during allocation or directory modification can leave mismatches among bitmaps, inode pointers, link counts and directory entries. The superblock records clean/dirty or error state, and e2fsck can perform a structured scan of allocation, namespace and connectivity to restore consistency when evidence permits.

Recovery cannot assume bitmap truth, because the bitmap may have been updated before a related inode pointer. An offline checker cross-correlates group descriptors, metadata reservations, block maps, inode ownership and directory reachability. Feature flags prevent old implementations from writing to formats they do not safely understand. A copied superblock alone does not establish that all file content survived.

EXT2 - Second Extended Filesystem

DESIGN RATIONALE AND LIMITS

Ext2’s block-group locality, separate inode namespace and multi-level addressing improved on floppy-oriented FAT and early Unix layouts. Its metadata remains comparatively understandable and has been documented for decades. The costs are possible fragmentation, indirect-block overhead for large files, and a potentially long full filesystem check after an unclean shutdown.

The absence of journalling can reduce write amplification under some workloads but is not equivalent to crash resilience. An ext2 disk may use extensions affecting directory structure or inode layout; the term EXT2 is not permission to ignore the feature masks. Kernel drivers, ext2 tools and modern compatibility layers should always validate block arithmetic and byte-order fields before writes.

EXT2 - Second Extended Filesystem

INFILTRATOROS INTEGRATION BOUNDARY

EXT2.so is an independent interpretation of ext2, not the Linux kernel ext2 driver. Its supported subset of writes and directory operations must be documented from tests. A native ELF shared library is not automatically a boot filesystem or a Linux VFS implementation; those require separate adapter and early-boot qualification.

Original ext2 superblocks, inodes, bitmaps and directory records are the primary objects of this guide. The published ‘.ko‘ and portable ‘.deb‘ are optional compatibility products, and verification of their package hashes is independent of integrity of user data on an ext2 volume.

EXT2 - Second Extended Filesystem

SOURCES AND PRIMARY REFERENCES

The documents below are the historical and engineering reference points used for this guide. Where an on-media field or feature flag is ambiguous, consult the original format documentation and its version-specific implementation notes before interpreting or changing media.

1. Linux kernel: The Second Extended Filesystem

https://kernel.org/doc/html/v6.6/filesystems/ext2.html

2. Linux kernel EXT4 overview, evolution of ext-family block groups

https://docs.kernel.org/filesystems/ext4/overview.html

The InfiltratorOS shared-object module, optional kernel adapter, native builder and release-specific tests are a separate implementation. A successful document build or package installation does not qualify physical writes, recovery, kernel mounting or boot.

EXT2 / technical detail

ON-DISK FIELD AND STRUCTURE INVENTORY

Format-specific field names and meanings are summarised below; where applicable the original specification defines exact endianness, offsets, checksums and format-revision limits.

1. Superblock: at filesystem offset 1024 bytes.

2. s_log_block_size: data block geometry.

3. s_feature_compat, incompat, ro_compat: version gates.

4. Group Descriptor Table: bitmap and inode table positions.

5. Block bitmap: allocated physical data and metadata blocks.

6. Inode bitmap: persistent inode availability.

7. Inode: direct and multiple indirect data block pointers.

8. Directory entries: inode number, record length, name data.

EXT2 / technical detail

HISTORICAL FORMAT DEVELOPMENT

Selected design milestones explain why this format acquired its present layout. Dates refer to historical format or implementation changes, not InfiltratorOS releases.

1. Early 1990s: Linux moves past Minix filesystem constraints.

2. January 1993: Second Extended Filesystem released.

3. Late 1990s: ext2 becomes a common Linux default.

4. 1999-2001: ext3 adds journalling to the same format family.

Important historical distinction: newly introduced driver features are not necessarily changes to the original on-disk format. Read the version or feature bits rather than assuming current software support defines historic media.

EXT2 / technical detail

FORMAT VALIDATION AND RECOVERY INVARIANTS

A writable implementation must establish all of these conditions before modifying any original-format media. These checks describe the minimum class of invariants, not proof of complete recovery qualification.

1. Validate all feature bits before selecting write mode.

2. Check metadata blocks are reserved and not allocated as file data.

3. Cross-check inode and block bitmaps with inode pointers.

4. Verify directory reachability and hard-link counts.

5. Handle sparse-super replicas and correct inode record size.

Forensic principle: use a separate verified image, retain original structures, and report conflicting evidence instead of inventing a new supposedly clean filesystem state.

EXT2 / engineering detail

DISK READ PATH / ENGINEERING NOTES

1. Read the primary superblock at byte offset 1024, then validate the block-size shift, inode size, block count and group descriptor geometry.

2. Locate block group descriptors and the corresponding inode and block bitmaps. Superblock and descriptor backup placement depend on filesystem features.

3. Compute inode number to group and index, then locate its inode-table entry. Inode number two is the root directory under the standard layout.

4. Interpret direct and single, double or triple indirect pointers with the block size and correct endianness. Do not treat each pointer as an extent length.

5. Decode variable-length directory records with record length and name length; check that each record advances safely within a directory block.

6. Map hard links through inode reference counts. Multiple directory names can refer to one inode while symbolic links have separate inode data semantics.

7. Check filesystem feature masks and revision-dependent inode extensions before accessing extended attributes, large files or other optional metadata.

8. Treat the lack of an ext2 journal as a defining consistency characteristic. The allocator relies on bitmap, directory and inode coherence after an interruption.

EXT2 / engineering detail

VALIDATION, RECOVERY AND COMPATIBILITY

1. Check group descriptor and bitmap bounds before trusting an inode number. A plausible superblock does not make every group readable.

2. Reject directory entries with zero progress, invalid record sizes or inode references outside the recorded inode population.

3. Validate indirect block levels and detect loops, duplicate block ownership and pointers outside the volume; corrupted trees must not trigger infinite traversal.

4. Cross-check inode link counts with directory references and allocated data blocks with the block bitmap. Disagreements require diagnosis, not automatic rewriting.

5. Respect clean-state and error flags. A volume modified without journaling may need a full offline consistency check before writes.

6. Do not assume all historical ext2 filesystems have the same inode size, revision or feature flags; the on-disk metadata controls interpretation.

7. Where superblock copies disagree, report their locations and revisions. Selecting the most convenient copy is not a safe repair algorithm.

8. A writer must order inode, directory and allocation updates conservatively, because ext2 does not provide a transactional replay record.